Assessing the impact of the digitisation era on the evolution of Edinburgh's finance sector

B273700

1 Overview

This analysis aims to assess the impact of the 2000 Dot-Com crash as a turning point on the decline of Edinburgh's finance sector, as this event led to rapid digitisation of industry across the world. We use temporal and spatial analysis to determine which regions and which sub-sectors were most heavily affected, if at all. We use the Edinburgh City Council Shop Survey to inform these findings, which span the period from 1986 to 2015 and include brick-and-mortar financial businesses, our primary focus. Our analysis revealed that the rate of contraction of the finance sector tripled after the Dot-Com crash, leading to every ward in Edinburgh having either lost financial business or stagnated in their growth, with one exception. These findings support the view that digitisation had a negative effect on Edinburgh's brick and mortar finance sector.

2 Introduction

Context and motivation The "dot-com era" (1995- 2000) was a period in which internet based companies became significantly overvalued leading to a "burst" in their companies' valuation[1]. This event transformed the financial landscape nationally through the widespread adoption of internet banking and subsequent financial technology advancements, such as the first mobile banking app[5], despite short term recession. This digitisation of the banking system and financial services as a whole, has led to the closure of around 53% of banks from the 1990s to 2016 as financial services move to more profitable online methods[2]. This study will examine the impact of this digitisation of bank services and the wider financial sector of Edinburgh through Edinburgh City Council's "Shop Survey" Dataset to determine if this digitisation of the financial market has an impact on the number of brick-and-mortar financial services in the city wide area, as well as each city area.

Previous work One economic paper, written by Minhae Kim, named "Does the Internet replace Brick-and-Mortar Bank Branches?"[3] assesses the question of whether the internet is a complement for brick-and-mortar banks, or if it is a substitute for physical bank branches within an American context. This study found that the more people use the internet, the larger the market for banks, which thereby increases brick and mortar locations. A report written by the Federation of Small Businesses, named "Locked Out - The Impact Of Bank Branch Closures On Small Businesses"[2] investigates the impact of Bank Branch closures on small businesses. This study found that around 53% of bank branches have closed since the 1990s, coinciding with the rise of online banking. Furthermore, the report written by Nicole Winchester in the Lords Library, titled "Closure of bank branches: Impact on rural communities"[6] states that nationally bank branches have significantly decreased since the 1980s, due to the rise of mobile banking, stating that 74% of people now interact with banks outside of the Brick-and-Mortar bank branches.

Objectives The objectives of this analysis are to determine to what extent the city of Edinburgh and Its regions have been affected by the digitalisation of banks and wider financial services. This will be achieved by analysing the number of banks, as well as other services such as financial consultancy and

building societies, as well as the spatial concentration of these, to measure how each region of the city has been impacted using visual and statistical analysis of the Edinburgh Shop Survey.

3 Data

Data provenance The data for this project has been obtained through the "Edinburgh City Council Open Spatial Data Portal", in the Shop Survey dataset[7]. This data set is accessible and will be downloaded as CSV files. This data is licenced under the Open Government License v3.0, which entitles us to copy, publish and transmit the information, as well as adapt the information as long as we acknowledge the source of the information[4].

Data description The data for this analysis has come in the form of 5 separate datasets in CSV form, each dating from the periods: 1986, 1996, 2004, 2010, 2015. Each of these datasets are lengths of 7,396, 7,668, 7,123, 7,123, and 7,396 for each period respectively. Each of these datasets contain similar columns, such as: *X, Y, DESCRIP, ADDRESS, POSTCODE, OBJECTID, BUSINESS*, as well as *USECASE86, USECASE96, USECASE04, USECASE10* for each dataset between 1986 and 2010. The 2015 dataset is formatted differently since there are no other year's use cases, as there is only *USECASE15*, the column *POSTCODE* is renamed to *CVR15_POST* and there is the addition of the *EASTING, NORTHING* columns.

Data processing For us to analyse these datasets accurately, we firstly clean the data inside the Shop Survey dataset by removing entries that cannot be used or will incorrectly skew. We do this by removing values from the table that have a blank *DESCRIP* value, which removed 2 values, and we also checked for duplicate values but 0 were found. In addition to this, there are entries in the shop dataset that do not entail actual shops, such as: "Vacant", "UNKNOWN USE", "demolished" and residential, under the *USECASE* column, which we have removed from the dataset. This removed 3,725 values in total from each of the 5 datasets. Furthermore, in the effort of standardising our descriptions for our tables for the purpose filtering later on, we have formatted each value in the *DESCRIP* column to be solely lowercase, thereby ensuring consistent filtering. Since the data came in 5 files for each year (1986, 1996, 2004, 2010, 2015), to analyse the entire dataset, once the dataset is cleaned, each dataset can be concatenated into one Dataframe, FinanceDF, which contains X and Y coordinates, description, useclass, Year of business, ward and our defined Region, which is discussed further in Exploration and Analysis. This dataframe contains every shop associated with the finance sector in Edinburgh. In addition to this Dataframe, we also create another comparative dataframe FoodDF, which contains identical columns, and 3 dataframes representing the 3 sub-sectors Banking, Financial Services and Financial Products, which were formed by filtering FinanceDF by a set of masks representing businesses under these respective sub-sectors.

4 Exploration and analysis

To evaluate how the Dot-Com crash and the corresponding digitisation age, we can assess the change in brick and mortar financial businesses in the Edinburgh City Council administrative area, including spatial and temporal change.

To understand the affect of the Dot-Com crash and the subsequent digitisation on the finance sector in Edinburgh, we can observe how 3 major subsections of the finance sector evolve in each area of the city. We can do this by adding each of Edinburgh's wards into 5 distinct regions, central (Morningside, City Centre, Southside / Newington, Fountainbridge / Craiglockhart), north (Leith, Leith Walk, Forth, Inverleith), west (Almond, Drum Brae / Gyle, Corstorphine / Murrayfield, Sighthill / Gorgie), south (Liberton / Gilmerton, Colinton / Fairmilehead, Pentland Hills) and east (Portobello / Craigmillar,

"Craigentinny / Duddingston"). This is done so that spatial patterns can be easier seen as can be seen below.

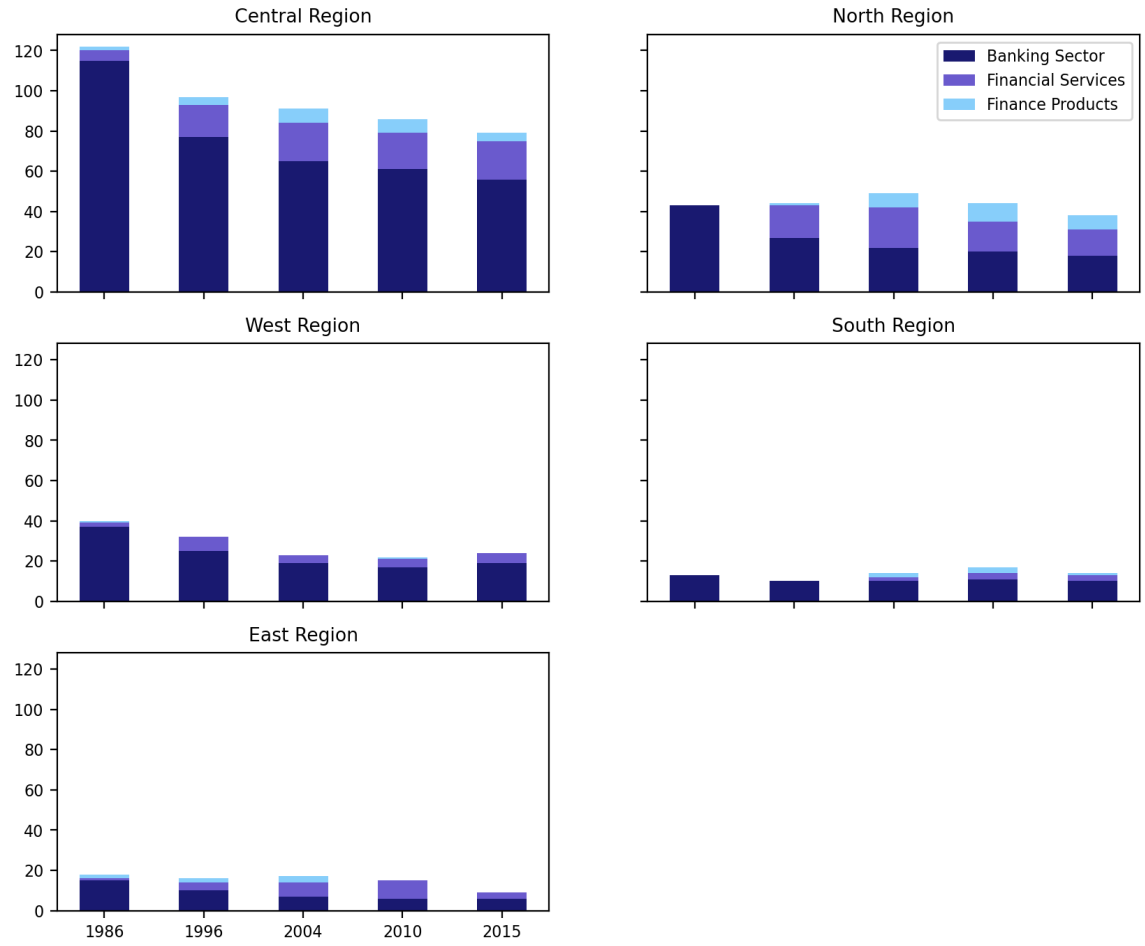


Figure 1: Evolution of Edinburgh's Financial sub sectors by region (1986 - 2015).

As we can see in Figure 1, a significant portion of the Financial sector in 1986 were considered under the banking sector, the total cumulative number of these banks decreases across all regions in Edinburgh over the time period, with the most significant decrease of this period within the central region declining from 120 to 80, however this region still contains the most cumulative Financial Businesses by 2015, which presents that for the most part the financial sector remained spatially central throughout the time period. Additionally, other regions such as the Northern, Southern and Western regions experienced stagnation overall over the period, with a pattern of replacing replacing banking sector businesses with financial services such as accounting. These changes align with the digitisation era, as businesses such as banks move to digital services, which removes the need for banks for most cases such as withdrawals and transfers.

While it can be seen that the cumulative financial sector count decreases over multiple regions, this could be caused by another external factor unrelated to the digitisation of Businesses, such as a general decline in business across the city. To reinforce that the reduction of businesses in the financial sector is a consequence of the Dot-Com crash and subsequent digitisation of media and not of other factors, we use another sector within the city, the food and drink sector, to assess their growth in comparison. We choose this industry as the food industry is robust to digitisation and thereby should show a contrary growth trend if the digitisation of businesses is a factor.

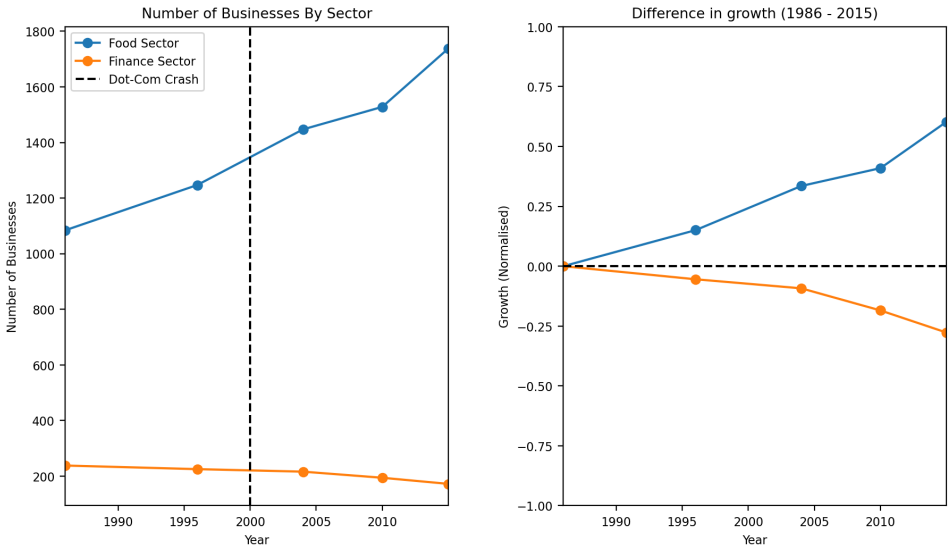


Figure 2: Comparative analysis of Business Count of Food and Financial sectors in Edinburgh (1986 - 2015), Normalised difference in growth between Food and Financial sectors.

As can be seen in Figure 2, the food sector diverges from the financial sectors difference in growth, as the food sector is seen as consistently growing in business numbers between 1986-2015, growing by 60% during the period, while the growth rate of the financial sector contracted by 25%. While this does not explicitly prove that the financial sector specifically declined due to digitisation of financial services, but it disproves that a city-wide economic slump occurred which pushed the financial sector into decline.

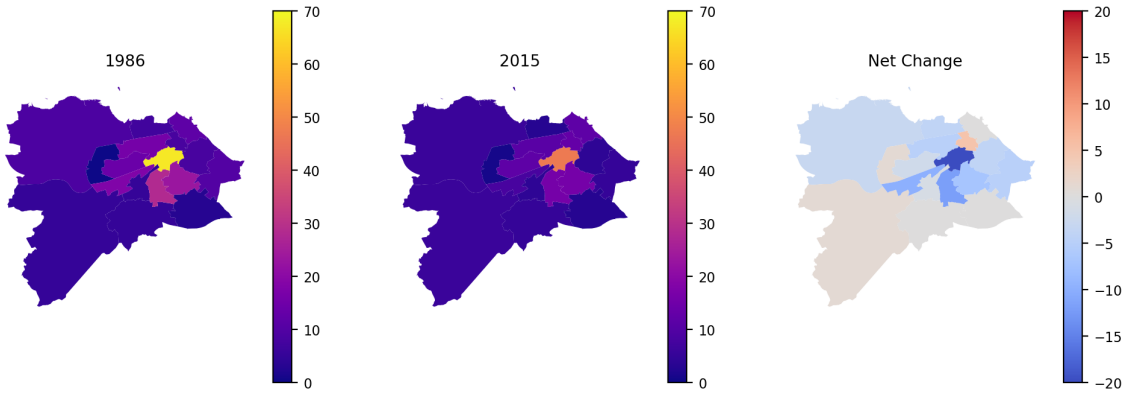


Figure 3: Choropleth Map of Financial Business frequency by Edinburgh Administrative Ward (1986 - 2015), and Net Change between 1986 and 2015.

This decline of financial businesses in the city can be visualised through Figure 3, as every ward in Edinburgh experienced either decline or stagnation, the most notable of these results being the city centre ward, which lost 20 businesses in the period. This Figure displays that Edinburgh's financial sector experienced disappearance rather than a spatial shift between wards, displaying that no centralisation/decentralisation occurred.

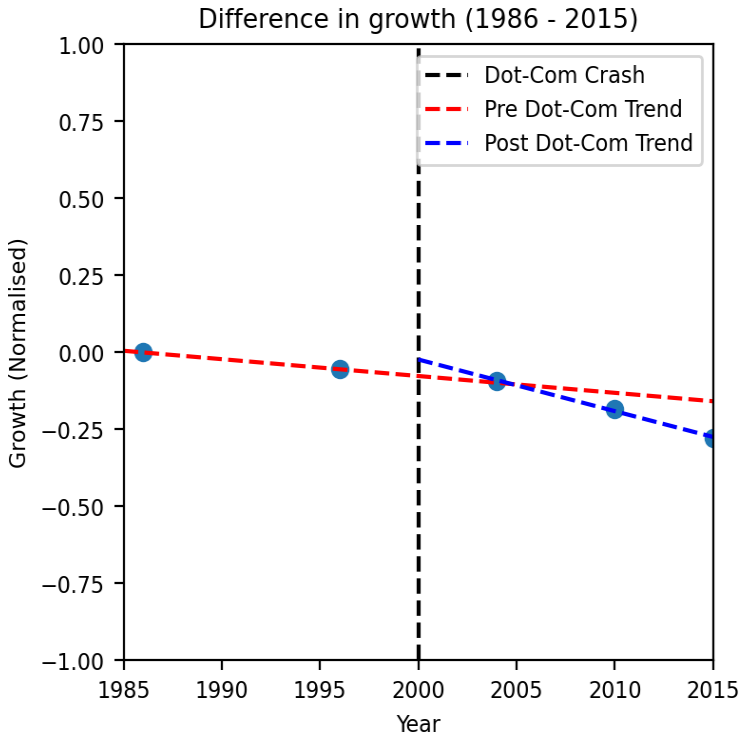


Figure 4: Piecewise Regression graph presenting changing Financial Sector growth trends pre and post Dot-Com era.

Since we have found that there is financial sector specific decline in business, we can utilise a piece wise regression analysis, with segments before and after the Dot-Com crash to determine if this crash is the focal point of the decline in growth of the financial sector in Edinburgh, which we can use since our data points present linearity. Figure 4 presents that between 1986 and 2000, then 2000 and 2015, the rate of contraction tripled in the Post Dot-Com Crash era, which can be seen in their associated β_1 coefficients -0.00546 (Pre Dot-Com) and -0.01676 (Post Dot-Com). It is important to note that this data could be inaccurate due to the limited data points (5) given by the Edinburgh Shop Survey.

5 Discussion and conclusions

Summary of findings The results of this study show a variety of both temporal and spatial results. Overall, the financial sector in Edinburgh faced a sustained contraction during the period 1986-2015, however it can be seen that the rate of contraction increased to more than triple after the Dot-Com crash of 2000, contracting to a maximum of 75% of the original Financial Sector. Furthermore, every region and ward with one exception either stagnated or lost finance business in their respective location, signalling a possible move to online methods of finances, but the central region (city centre ward in particular) remained the 'hub' of finance sector businesses. Additionally, we found that the finance sector contraction in Edinburgh was sector specific, outlining positive trends in the food sector which can be considered more resilient to digitalisation.

Evaluation of own work: strengths and limitations One of the main limitations of this financial sector analysis is the number of years supplied, as well as the amount of time elapsed between years. This makes it difficult to develop a larger scope of the financial sectors evolution in Edinburgh, and makes readings such as in the regression less reliable as a forecast of past contraction and of future contractions. Furthermore, this makes it hard to distinguish which economic events had a more significant impact, since multiple economic events, most notably the 2008 financial crisis also occurred during this period

with no direct data point to address the impact of this independently of the digitalisation era as a whole. In contrast, one strength of this analysis is the comprehensive breakdown of Edinburgh into regions and its administrative boundaries, as this provides multiple levels of interpretation on Edinburgh's unique financial environment.

Comparison with any other related work Other related work correspond with my analysis results, while others contradict these. One example is the Federation of Small Businesses report, as our Figures such Figure one present the significant decline in the banking sector in particular, however our analysis presents the largest loss of financial services in the central regions rather than the more rural regions such as the south or east regions. In contrast, another study by Minhae Kim found that there is a positive relationship between internet usage and brick-and-mortar banking locations, however this study was taken in America and therefore there may be other external factors that have guided my results.

Improvements and extensions One way in which we can improve this analysis of the evolution of financial sector in Edinburgh is to incorporate more datasets which provide a fuller image of the financial sector in Edinburgh, since offices containing large financial companies are not represented within the dataset, as well as virtual companies registered within Edinburgh which would also provide a valuable insight into the extent of finance digitisation.

References

[1] Brian Duignan. *"Dot-Com bubble"*. https://www.britannica.com/money/dot-com-bubble. (Visited on 23/03/2026).

[2] Federation of Small Businesess. *Locked out: the impact of bank closures on small businesses*. https://www.fsb.org.uk/resources/policy-reports/locked-out-MCAWGDK2SR3NACFFETC7K6H7LQWU. (Visited on 24/03/2026).

[3] Minhae Kim. *Does the Internet Replace Brick-and-Mortar Bank Branches?* https://minhaekim.org/research/Kim_internet_and_bank_branches.pdf. (Visited on 24/03/2026).

[4] National Archives. *Open Government License*. https://www.nationalarchives.gov.uk/doc/open-government-licence/version/3/. (Visited on 25/03/2026).

[5] Natwest Group. *Banking app, 2011*. https://www.natwestgroup.com/heritage/history-100/objects-by-theme/going-the-extra-mile/banking-app-2011.html. (Visited on 23/03/2026).

[6] Nicole Winchester. *Closure of bank branches: Impact on rural communities*. https://lordslibrary.parliament.uk/closure-of-bank-branches-impact-on-rural-communities/. (Visited on 24/03/2026).

[7] The City of Edinburgh Council. *Shop Survey*. https://city-of-edinburgh-council-open-spatial-data-cityofedinburgh.hub.arcgis.com/search?tags=retail. (Visited on 25/03/2026).

FDS-Project-Code

April 3, 2026

1 Imports

[1]:
```
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
import geopandas as gpd
from shapely.geometry import Point
import numpy as np
import warnings
mpl.rcParams["font.size"] = 8
mpl.rcParams['figure.dpi'] = 200

#does not bypass any warnings or errors it ignores future warnings of␣
 ↪depreciation
warnings.simplefilter(action='ignore', category=FutureWarning)
```

2 Data

[2]:
```
 #importing each of the 5 respective shop surveys

DF1986 = pd.read_csv("Datasets/Shop_Survey_1986.csv")
DF1996 = pd.read_csv("Datasets/Shop_Survey_1996.csv")
DF2004 = pd.read_csv("Datasets/Shop_Survey_2004.csv")
DF2010 = pd.read_csv("Datasets/Shop_Survey_2010.csv")
DF2015 = pd.read_csv("Datasets/Shop_Survey_2015.csv")
```

[3]:
```
#importing Edinburgh Ward Boundaries

wards = gpd.read_file("Datasets/Edinburgh_Wards.geojson")
wards = wards.to_crs("EPSG:27700")
```

3 Exploratory Data Analysis

[4]:
```
#Drop each of the needless columns from each of the Dataframes
#(We don't need each other USECLASS other than the year since every instance of␣
 ↪a business is in the other years)

DF1986.drop(["BUSINESS86", "USECLASS96", "USECLASS04", "USECLASS10", "ADDRESS",␣
 ↪"OBJECTID", "POSTCODE"], axis=1, inplace=True)
DF1996.drop(["BUSINESS96", "USECLASS86", "USECLASS04", "USECLASS10", "ADDRESS",␣
 ↪"OBJECTID", "POSTCODE"], axis=1, inplace=True)
DF2004.drop(["BUSINESS04", "USECLASS86", "USECLASS96", "USECLASS10", "ADDRESS",␣
 ↪"OBJECTID", "POSTCODE"], axis=1, inplace=True)
DF2010.drop(["BUSINESS10", "USECLASS86", "USECLASS96", "USECLASS04", "ADDRESS",␣
 ↪"OBJECTID", "POSTCODE"], axis=1, inplace=True)
DF2015.drop(["BUSINESS15", "ADDRESS", "EASTING", "NORTHING", "OBJECTID",␣
 ↪"CVR15_POST"], axis=1, inplace=True)
```

[5]:
```
#Rename each of the required columns to the same name so that we can␣
 ↪concatenate the Datadrames

DF1986 = DF1986.rename(columns={"USECLASS86": "Useclass", "DESCRIP86":␣
 ↪"Description"})
DF1996 = DF1996.rename(columns={"USECLASS96": "Useclass", "DESCRIP96":␣
 ↪"Description"})
DF2004 = DF2004.rename(columns={"USECLASS04": "Useclass", "DESCRIP04":␣
 ↪"Description"})
DF2010 = DF2010.rename(columns={"USECLASS10": "Useclass", "DESCRIP10":␣
 ↪"Description"})
DF2015 = DF2015.rename(columns={"USECLASS15": "Useclass", "DESCRIP15":␣
 ↪"Description"})
```

[6]:
```
#Create a year data point for every entry in each respective year, so that we␣
 ↪don't lose the year when we concat

DF1986["Year"] = [1986] * len(DF1986)
DF1996["Year"] = [1996] * len(DF1996)
DF2004["Year"] = [2004] * len(DF2004)
DF2010["Year"] = [2010] * len(DF2010)
DF2015["Year"] = [2015] * len(DF2015)
```

[7]:
```
#Concatenates each of the dataframes into one large Shop dataframe with␣
 ↪"Useclass", "Description", "X", "Y" (coordinates)
# and Year of the business existing

DFYears = [DF1986, DF1996, DF2004, DF2010, DF2015]
ShopsDF = pd.concat(DFYears)
```

```
ShopsDF = ShopsDF.reset_index(drop=True)
```

[8]:
```
#Replaces entries with no value to NaN values to drop later

ShopsDF["Useclass"].replace('', np.nan, inplace=True)
ShopsDF["Description"].replace('', np.nan, inplace=True)
```

```
/tmp/ipykernel_802/1388760371.py:1: FutureWarning: A value is trying to be set
on a copy of a DataFrame or Series through chained assignment using an inplace
method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  ShopsDF["Useclass"].replace('', np.nan, inplace=True)
/tmp/ipykernel_802/1388760371.py:2: FutureWarning: A value is trying to be set
on a copy of a DataFrame or Series through chained assignment using an inplace
method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  ShopsDF["Description"].replace('', np.nan, inplace=True)
```

[9]:
```
#Drops entries if an entry has a NaN value in the Useclass or Description,␣
 ↪since these are crucial to identifying
#businesses

ShopsDF.dropna(subset=["Useclass"], inplace=True)
ShopsDF.dropna(subset=["Description"], inplace=True)
```

[10]:
```
#Drops elements that are irrelevant to this dataset, since they're not␣
 ↪businesses

ShopsDF = ShopsDF.loc[~(ShopsDF["Useclass"].isin(["Vacant", "UNKNOWN USE",␣
 ↪"Demolished", "residential", "under refurb"]))]
```

[11]:
```
#make description lower case to make masking easier

ShopsDF["Description"] = ShopsDF["Description"].str.lower()
```

[12]:
```
#generate every food sector business from Shops

FoodMask = "public house|food|bar"
FoodFilter = ShopsDF["Useclass"].isin(["Class 3"]) | (ShopsDF["Useclass"].
 ↪isin(["SUI GENERIS"]) & ShopsDF["Description"].str.contains(FoodMask,␣
 ↪na=False))
```

[13]:
```
#uses masks to generate every food business for all 5 years

Food1986 = ShopsDF.loc[FoodFilter & ShopsDF["Year"].isin([1986])]
Food1996 = ShopsDF.loc[FoodFilter & ShopsDF["Year"].isin([1996])]
Food2004 = ShopsDF.loc[FoodFilter & ShopsDF["Year"].isin([2004])]
Food2010 = ShopsDF.loc[FoodFilter & ShopsDF["Year"].isin([2010])]
Food2015 = ShopsDF.loc[FoodFilter & ShopsDF["Year"].isin([2015])]
```

[14]:
```
#concatenate each of the years into one Food sector table
FoodFrames = [Food1986, Food1996, Food2004, Food2010, Food2015]
FoodDF = pd.concat(FoodFrames)
FoodDF = FoodDF.reset_index(drop=True)
```

[ ]:
```
#generate every finance sector and sub sector business
FinanceMask = "bank|building society|lender|financial|finance|bldg␣
 ↪soc|accountant|accounting|loans|insurance|build.soc.|bureau de␣
 ↪change|pensions|credit|mortgage|currency"
FinancialServicesMask = "financial|finance|accountant|accounting|bureau de␣
 ↪change|currency|credit"
FinanceProductsMask = "money|loans|insurance|mortgage"
BankingMask = "bank|building society|bldg soc|build.soc."

FinanceFilter = (ShopsDF["Useclass"].isin(["Class 2"])) &␣
 ↪ShopsDF["Description"].str.contains(FinanceMask)
```

[17]:
```
#generate every financial business for every year

Financial1986 = ShopsDF.loc[FinanceFilter & ShopsDF["Year"].isin([1986])]
Financial1996 = ShopsDF.loc[FinanceFilter & ShopsDF["Year"].isin([1996])]
Financial2004 = ShopsDF.loc[FinanceFilter & ShopsDF["Year"].isin([2004])]
Financial2010 = ShopsDF.loc[FinanceFilter & ShopsDF["Year"].isin([2010])]
Financial2015 = ShopsDF.loc[FinanceFilter & ShopsDF["Year"].isin([2015])]
```

[18]:
```
#concatenate each of these finance businesses
```

```
FinanceFrames = [Financial1986, Financial1996, Financial2004, Financial2010,␣
 ↪Financial2015]
FinanceDF = pd.concat(FinanceFrames)
FinanceDF = FinanceDF.reset_index(drop=True)
```

[19]:
```
#use each of the X Y values from shop to map each finance business to a point␣
 ↪system

geometry = [Point(xy) for xy in zip(FinanceDF.X, FinanceDF.Y)]
points = gpd.GeoDataFrame(FinanceDF, crs="EPSG:27700", geometry=geometry)
```

[20]:
```
#associate businesses with each particular ward and drop now unneeded columns
FinancialWard = gpd.sjoin(points, wards, predicate='within')
FinancialWard.drop(["X", "Y", "index_right", "OBJECTID", "Link", "Date",␣
 ↪"Ward_Code", "Ward_No"], axis=1, inplace=True)
```

[21]:
```
FinanceWardFrames = [FinancialWard]
FinanceWardDF = pd.concat(FinanceWardFrames)
```

[22]:
```
#function that maps every ward to one of 5 "regions" in edinburgh.

def toRegion(Ward):
    central = ["Morningside", "City Centre", "Southside / Newington",␣
 ↪"Fountainbridge / Craiglockhart"]
    north = ["Leith", "Leith Walk", "Forth", "Inverleith"]
    west = ["Almond", "Drum Brae / Gyle", "Corstorphine / Murrayfield",␣
 ↪"Sighthill / Gorgie"]
    south = ["Liberton / Gilmerton", "Colinton / Fairmilehead", "Pentland␣
 ↪Hills"]
    east = ["Portobello / Craigmillar", "Craigentinny / Duddingston"]
    if Ward in central:
        return "Central"
    elif Ward in north:
        return "North"
    elif Ward in west:
        return "West"
    elif Ward in south:
        return "South"
    elif Ward in east:
        return "East"
```

[ ]:
```
# creates new column region and ward name in Finance and associates these with␣
 ↪each business
FinanceDF["Region"] = FinanceWardDF["Ward_Name"].apply(toRegion)

FinanceDF["Ward Name"] = FinanceWardDF["Ward_Name"]
```

[25]:
```
#Mask to filter Banking from the finance businesses
BankingFilter = (FinanceDF["Description"].str.contains(BankingMask, na=False))␣
↪& ~(FinanceDF["Description"].str.contains(FinancialServicesMask + '|' +␣
↪FinanceProductsMask, na=False))
```

[26]:
```
#generate dataframes for every year

Banking1986 = FinanceDF.loc[BankingFilter & FinanceDF["Year"].isin([1986])]
Banking1996 = FinanceDF.loc[BankingFilter & FinanceDF["Year"].isin([1996])]
Banking2004 = FinanceDF.loc[BankingFilter & FinanceDF["Year"].isin([2004])]
Banking2010 = FinanceDF.loc[BankingFilter & FinanceDF["Year"].isin([2010])]
Banking2015 = FinanceDF.loc[BankingFilter & FinanceDF["Year"].isin([2015])]
```

[27]:
```
#concatenate each bank to one table

BankingFrames = [Banking1986, Banking1996, Banking2004, Banking2010,␣
↪Banking2015]
BankingDF = pd.concat(BankingFrames)
BankingDF = BankingDF.reset_index(drop=True)
```

[28]:
```
#these same steps for bank are applied to Financial services and Financial␣
↪products as follows

FinancialServicesFilter = (FinanceDF["Description"].str.
↪contains(FinancialServicesMask, na=False)) & ~(FinanceDF["Description"].str.
↪contains(BankingMask + '|' + FinanceProductsMask, na=False))
```

[29]:
```
FinancialServices1986 = FinanceDF.loc[FinancialServicesFilter &␣
↪FinanceDF["Year"].isin([1986])]
FinancialServices1996 = FinanceDF.loc[FinancialServicesFilter &␣
↪FinanceDF["Year"].isin([1996])]
FinancialServices2004 = FinanceDF.loc[FinancialServicesFilter &␣
↪FinanceDF["Year"].isin([2004])]
FinancialServices2010 = FinanceDF.loc[FinancialServicesFilter &␣
↪FinanceDF["Year"].isin([2010])]
FinancialServices2015 = FinanceDF.loc[FinancialServicesFilter &␣
↪FinanceDF["Year"].isin([2015])]
```

[30]:
```
FinancialServicesFrames = [FinancialServices1986, FinancialServices1996,␣
↪FinancialServices2004, FinancialServices2010, FinancialServices2015]
FinancialServicesDF = pd.concat(FinancialServicesFrames)
```

[31]:
```
FinanceProductsFilter = (FinanceDF["Description"].str.
↪contains(FinanceProductsMask, na=False)) & ~(FinanceDF["Description"].str.
↪contains(BankingMask + '|' + FinancialServicesMask, na=False))
```

[32]:
```
FinanceProducts1986 = FinanceDF.loc[FinanceProductsFilter & FinanceDF["Year"].
↪isin([1986])]
FinanceProducts1996 = FinanceDF.loc[FinanceProductsFilter & FinanceDF["Year"].
↪isin([1996])]
FinanceProducts2004 = FinanceDF.loc[FinanceProductsFilter & FinanceDF["Year"].
↪isin([2004])]
FinanceProducts2010 = FinanceDF.loc[FinanceProductsFilter & FinanceDF["Year"].
↪isin([2010])]
FinanceProducts2015 = FinanceDF.loc[FinanceProductsFilter & FinanceDF["Year"].
↪isin([2015])]
```

[33]:
```
#created dataframe which sums each of the sub sectors for every year in the␣
↪survey

FinancialBusinessCount = pd.DataFrame()
FinancialBusinessCount["Year"] = [1986, 1996, 2004, 2010, 2015]
FinancialBusinessCount["Banking Sector"] = [len(Banking1986), len(Banking1996),␣
↪len(Banking2004), len(Banking2010), len(Banking2015)]
FinancialBusinessCount["Financial Services"] = [len(FinancialServices1986),␣
↪len(FinancialServices1996), len(FinancialServices2004),␣
↪len(FinancialServices2010), len(FinancialServices2015)]
FinancialBusinessCount["Finance Products"] = [len(FinanceProducts1986),␣
↪len(FinanceProducts1996), len(FinanceProducts2004),␣
↪len(FinanceProducts2010), len(FinanceProducts2015)]
FinancialBusinessCount["Financial Industry"] = [len(Financial1986),␣
↪len(Financial1996), len(Financial2004), len(Financial2010),␣
↪len(Financial2015)]
FinancialBusinessCount = FinancialBusinessCount.set_index("Year")
```

[34]:
```
#Separates each of the sectors by year into each of the 5 regions: Central␣
↪North East South and West

inancialRegionCount1986 = pd.DataFrame()
FinancialRegionCount1986["Sector"] = ["Banking", "Finance Products", "Financial␣
↪Services", "Total"]
FinancialRegionCount1986["Central"] = [Banking1986["Region"].str.
↪contains("Central", na=False, regex=True).sum(),␣
↪FinanceProducts1986["Region"].str.contains("Central", na=False, regex=True).
↪sum(), FinancialServices1986["Region"].str.contains("Central", na=False,␣
↪regex=True).sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("Central")␣
↪& FinanceDF["Year"].isin([1986])].sum()]
```

```
FinancialRegionCount1986["North"] = [Banking1986["Region"].str.
  ↪contains("North", na=False, regex=True).sum(), FinanceProducts1986["Region"].
  ↪str.contains("North", na=False, regex=True).sum(),␣
  ↪FinancialServices1986["Region"].str.contains("North", na=False, regex=True).
  ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("North") &␣
  ↪FinanceDF["Year"].isin([1986])].sum()]
FinancialRegionCount1986["East"] = [Banking1986["Region"].str.contains("East",␣
  ↪na=False, regex=True).sum(), FinanceProducts1986["Region"].str.
  ↪contains("East", na=False, regex=True).sum(),␣
  ↪FinancialServices1986["Region"].str.contains("East", na=False, regex=True).
  ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("East") &␣
  ↪FinanceDF["Year"].isin([1986])].sum()]
FinancialRegionCount1986["South"] = [Banking1986["Region"].str.
  ↪contains("South", na=False, regex=True).sum(), FinanceProducts1986["Region"].
  ↪str.contains("South", na=False, regex=True).sum(),␣
  ↪FinancialServices1986["Region"].str.contains("South", na=False, regex=True).
  ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("South") &␣
  ↪FinanceDF["Year"].isin([1986])].sum()]
FinancialRegionCount1986["West"] = [Banking1986["Region"].str.contains("West",␣
  ↪na=False, regex=True).sum(), FinanceProducts1986["Region"].str.
  ↪contains("West", na=False, regex=True).sum(),␣
  ↪FinancialServices1986["Region"].str.contains("West", na=False, regex=True).
  ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("West") &␣
  ↪FinanceDF["Year"].isin([1986])].sum()]

FinancialRegionCount1996 = pd.DataFrame()
FinancialRegionCount1996["Sector"] = ["Banking", "Finance Products", "Financial␣
  ↪Services", "Total"]
FinancialRegionCount1996["Central"] = [Banking1996["Region"].str.
  ↪contains("Central", na=False, regex=True).sum(),␣
  ↪FinanceProducts1996["Region"].str.contains("Central", na=False, regex=True).
  ↪sum(), FinancialServices1996["Region"].str.contains("Central", na=False,␣
  ↪regex=True).sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("Central")␣
  ↪& FinanceDF["Year"].isin([1996])].sum()]
FinancialRegionCount1996["North"] = [Banking1996["Region"].str.
  ↪contains("North", na=False, regex=True).sum(), FinanceProducts1996["Region"].
  ↪str.contains("North", na=False, regex=True).sum(),␣
  ↪FinancialServices1996["Region"].str.contains("North", na=False, regex=True).
  ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("North") &␣
  ↪FinanceDF["Year"].isin([1996])].sum()]
FinancialRegionCount1996["East"] = [Banking1996["Region"].str.contains("East",␣
  ↪na=False, regex=True).sum(), FinanceProducts1996["Region"].str.
  ↪contains("East", na=False, regex=True).sum(),␣
  ↪FinancialServices1996["Region"].str.contains("East", na=False, regex=True).
  ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("East") &␣
  ↪FinanceDF["Year"].isin([1996])].sum()]
```

```
FinancialRegionCount1996["South"] = [Banking1996["Region"].str.
  ↪contains("South", na=False, regex=True).sum(), FinanceProducts1996["Region"].
  ↪str.contains("South", na=False, regex=True).sum(),␣
  ↪FinancialServices1996["Region"].str.contains("South", na=False, regex=True).
  ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("South") &␣
  ↪FinanceDF["Year"].isin([1996])].sum()]
FinancialRegionCount1996["West"] = [Banking1996["Region"].str.contains("West",␣
  ↪na=False, regex=True).sum(), FinanceProducts1996["Region"].str.
  ↪contains("West", na=False, regex=True).sum(),␣
  ↪FinancialServices1996["Region"].str.contains("West", na=False, regex=True).
  ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("West") &␣
  ↪FinanceDF["Year"].isin([1996])].sum()]

FinancialRegionCount2004 = pd.DataFrame()
FinancialRegionCount2004["Sector"] = ["Banking", "Finance Products", "Financial␣
  ↪Services", "Total"]
FinancialRegionCount2004["Central"] = [Banking2004["Region"].str.
  ↪contains("Central", na=False, regex=True).sum(),␣
  ↪FinanceProducts2004["Region"].str.contains("Central", na=False, regex=True).
  ↪sum(), FinancialServices2004["Region"].str.contains("Central", na=False,␣
  ↪regex=True).sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("Central")␣
  ↪& FinanceDF["Year"].isin([2004])].sum()]
FinancialRegionCount2004["North"] = [Banking2004["Region"].str.
  ↪contains("North", na=False, regex=True).sum(), FinanceProducts2004["Region"].
  ↪str.contains("North", na=False, regex=True).sum(),␣
  ↪FinancialServices2004["Region"].str.contains("North", na=False, regex=True).
  ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("North") &␣
  ↪FinanceDF["Year"].isin([2004])].sum()]
FinancialRegionCount2004["East"] = [Banking2004["Region"].str.contains("East",␣
  ↪na=False, regex=True).sum(), FinanceProducts2004["Region"].str.
  ↪contains("East", na=False, regex=True).sum(),␣
  ↪FinancialServices2004["Region"].str.contains("East", na=False, regex=True).
  ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("East") &␣
  ↪FinanceDF["Year"].isin([2004])].sum()]
FinancialRegionCount2004["South"] = [Banking2004["Region"].str.
  ↪contains("South", na=False, regex=True).sum(), FinanceProducts2004["Region"].
  ↪str.contains("South", na=False, regex=True).sum(),␣
  ↪FinancialServices2004["Region"].str.contains("South", na=False, regex=True).
  ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("South") &␣
  ↪FinanceDF["Year"].isin([2004])].sum()]
FinancialRegionCount2004["West"] = [Banking2004["Region"].str.contains("West",␣
  ↪na=False, regex=True).sum(), FinanceProducts2004["Region"].str.
  ↪contains("West", na=False, regex=True).sum(),␣
  ↪FinancialServices2004["Region"].str.contains("West", na=False, regex=True).
  ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("West") &␣
  ↪FinanceDF["Year"].isin([2004])].sum()]
```

```
FinancialRegionCount2010 = pd.DataFrame()
FinancialRegionCount2010["Sector"] = ["Banking", "Finance Products", "Financial␣
 ↪Services", "Total"]
FinancialRegionCount2010["Central"] = [Banking2010["Region"].str.
 ↪contains("Central", na=False, regex=True).sum(),␣
 ↪FinanceProducts2010["Region"].str.contains("Central", na=False, regex=True).
 ↪sum(), FinancialServices2010["Region"].str.contains("Central", na=False,␣
 ↪regex=True).sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("Central")␣
 ↪& FinanceDF["Year"].isin([2010])].sum()]
FinancialRegionCount2010["North"] = [Banking2010["Region"].str.
 ↪contains("North", na=False, regex=True).sum(), FinanceProducts2010["Region"].
 ↪str.contains("North", na=False, regex=True).sum(),␣
 ↪FinancialServices2010["Region"].str.contains("North", na=False, regex=True).
 ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("North") &␣
 ↪FinanceDF["Year"].isin([2010])].sum()]
FinancialRegionCount2010["East"] = [Banking2010["Region"].str.contains("East",␣
 ↪na=False, regex=True).sum(), FinanceProducts2010["Region"].str.
 ↪contains("East", na=False, regex=True).sum(),␣
 ↪FinancialServices2010["Region"].str.contains("East", na=False, regex=True).
 ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("East") &␣
 ↪FinanceDF["Year"].isin([2010])].sum()]
FinancialRegionCount2010["South"] = [Banking2010["Region"].str.
 ↪contains("South", na=False, regex=True).sum(), FinanceProducts2010["Region"].
 ↪str.contains("South", na=False, regex=True).sum(),␣
 ↪FinancialServices2010["Region"].str.contains("South", na=False, regex=True).
 ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("South") &␣
 ↪FinanceDF["Year"].isin([2010])].sum()]
FinancialRegionCount2010["West"] = [Banking2010["Region"].str.contains("West",␣
 ↪na=False, regex=True).sum(), FinanceProducts2010["Region"].str.
 ↪contains("West", na=False, regex=True).sum(),␣
 ↪FinancialServices2010["Region"].str.contains("West", na=False, regex=True).
 ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("West") &␣
 ↪FinanceDF["Year"].isin([2010])].sum()]

FinancialRegionCount2015 = pd.DataFrame()
FinancialRegionCount2015["Sector"] = ["Banking", "Finance Products", "Financial␣
 ↪Services", "Total"]
FinancialRegionCount2015["Central"] = [Banking2015["Region"].str.
 ↪contains("Central", na=False, regex=True).sum(),␣
 ↪FinanceProducts2015["Region"].str.contains("Central", na=False, regex=True).
 ↪sum(), FinancialServices2015["Region"].str.contains("Central", na=False,␣
 ↪regex=True).sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("Central")␣
 ↪& FinanceDF["Year"].isin([2015])].sum()]
```

```
FinancialRegionCount2015["North"] = [Banking2015["Region"].str.
  ↪contains("North", na=False, regex=True).sum(), FinanceProducts2015["Region"].
  ↪str.contains("North", na=False, regex=True).sum(),␣
  ↪FinancialServices2015["Region"].str.contains("North", na=False, regex=True).
  ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("North") &␣
  ↪FinanceDF["Year"].isin([2015])].sum()]
FinancialRegionCount2015["East"] = [Banking2015["Region"].str.contains("East",␣
  ↪na=False, regex=True).sum(), FinanceProducts2015["Region"].str.
  ↪contains("East", na=False, regex=True).sum(),␣
  ↪FinancialServices2015["Region"].str.contains("East", na=False, regex=True).
  ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("East") &␣
  ↪FinanceDF["Year"].isin([2015])].sum()]
FinancialRegionCount2015["South"] = [Banking2015["Region"].str.
  ↪contains("South", na=False, regex=True).sum(), FinanceProducts2015["Region"].
  ↪str.contains("South", na=False, regex=True).sum(),␣
  ↪FinancialServices2015["Region"].str.contains("South", na=False, regex=True).
  ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("South") &␣
  ↪FinanceDF["Year"].isin([2015])].sum()]
FinancialRegionCount2015["West"] = [Banking2015["Region"].str.contains("West",␣
  ↪na=False, regex=True).sum(), FinanceProducts2015["Region"].str.
  ↪contains("West", na=False, regex=True).sum(),␣
  ↪FinancialServices2015["Region"].str.contains("West", na=False, regex=True).
  ↪sum(), FinanceDF.loc[FinanceDF["Region"].str.contains("Central") &␣
  ↪FinanceDF["Year"].isin([2015])].sum()]
```

[35]:
```
#Adds arrays for every region for every sub sector in finance

years = [1986, 1996, 2004, 2010, 2015]
BankingNumberCentral = np.array([Banking1986["Region"].str.contains("Central",␣
  ↪na=False, regex=True).sum(), Banking1996["Region"].str.contains("Central",␣
  ↪na=False, regex=True).sum(), Banking2004["Region"].str.contains("Central",␣
  ↪na=False, regex=True).sum(), Banking2010["Region"].str.contains("Central",␣
  ↪na=False, regex=True).sum(), Banking2015["Region"].str.contains("Central",␣
  ↪na=False, regex=True).sum()])
FinanceProductsNumberCentral = np.array([FinanceProducts1986["Region"].str.
  ↪contains("Central", na=False, regex=True).sum(),␣
  ↪FinanceProducts1996["Region"].str.contains("Central", na=False, regex=True).
  ↪sum(), FinanceProducts2004["Region"].str.contains("Central", na=False,␣
  ↪regex=True).sum(), FinanceProducts2010["Region"].str.contains("Central",␣
  ↪na=False, regex=True).sum(), FinanceProducts2015["Region"].str.
  ↪contains("Central", na=False, regex=True).sum()])
```

```
FinancialServicesNumberCentral = np.array([FinancialServices1986["Region"].str.
  ↪contains("Central", na=False, regex=True).sum(),␣
  ↪FinancialServices1996["Region"].str.contains("Central", na=False,␣
  ↪regex=True).sum(), FinancialServices2004["Region"].str.contains("Central",␣
  ↪na=False, regex=True).sum(), FinancialServices2010["Region"].str.
  ↪contains("Central", na=False, regex=True).sum(),␣
  ↪FinancialServices2015["Region"].str.contains("Central", na=False,␣
  ↪regex=True).sum()])

BankingNumberNorth = np.array([Banking1986["Region"].str.contains("North",␣
  ↪na=False, regex=True).sum(), Banking1996["Region"].str.contains("North",␣
  ↪na=False, regex=True).sum(), Banking2004["Region"].str.contains("North",␣
  ↪na=False, regex=True).sum(), Banking2010["Region"].str.contains("North",␣
  ↪na=False, regex=True).sum(), Banking2015["Region"].str.contains("North",␣
  ↪na=False, regex=True).sum()])
FinanceProductsNumberNorth = np.array([FinanceProducts1986["Region"].str.
  ↪contains("North", na=False, regex=True).sum(), FinanceProducts1996["Region"].
  ↪str.contains("North", na=False, regex=True).sum(),␣
  ↪FinanceProducts2004["Region"].str.contains("North", na=False, regex=True).
  ↪sum(), FinanceProducts2010["Region"].str.contains("North", na=False,␣
  ↪regex=True).sum(), FinanceProducts2015["Region"].str.contains("North",␣
  ↪na=False, regex=True).sum()])
FinancialServicesNumberNorth = np.array([FinancialServices1986["Region"].str.
  ↪contains("North", na=False, regex=True).sum(),␣
  ↪FinancialServices1996["Region"].str.contains("North", na=False, regex=True).
  ↪sum(), FinancialServices2004["Region"].str.contains("North", na=False,␣
  ↪regex=True).sum(), FinancialServices2010["Region"].str.contains("North",␣
  ↪na=False, regex=True).sum(), FinancialServices2015["Region"].str.
  ↪contains("North", na=False, regex=True).sum()])

BankingNumberWest = np.array([Banking1986["Region"].str.contains("West",␣
  ↪na=False, regex=True).sum(), Banking1996["Region"].str.contains("West",␣
  ↪na=False, regex=True).sum(), Banking2004["Region"].str.contains("West",␣
  ↪na=False, regex=True).sum(), Banking2010["Region"].str.contains("West",␣
  ↪na=False, regex=True).sum(), Banking2015["Region"].str.contains("West",␣
  ↪na=False, regex=True).sum()])
FinanceProductsNumberWest = np.array([FinanceProducts1986["Region"].str.
  ↪contains("West", na=False, regex=True).sum(), FinanceProducts1996["Region"].
  ↪str.contains("West", na=False, regex=True).sum(),␣
  ↪FinanceProducts2004["Region"].str.contains("West", na=False, regex=True).
  ↪sum(), FinanceProducts2010["Region"].str.contains("West", na=False,␣
  ↪regex=True).sum(), FinanceProducts2015["Region"].str.contains("West",␣
  ↪na=False, regex=True).sum()])
```

```
FinancialServicesNumberWest = np.array([FinancialServices1986["Region"].str.
  ↪contains("West", na=False, regex=True).sum(),␣
  ↪FinancialServices1996["Region"].str.contains("West", na=False, regex=True).
  ↪sum(), FinancialServices2004["Region"].str.contains("West", na=False,␣
  ↪regex=True).sum(), FinancialServices2010["Region"].str.contains("West",␣
  ↪na=False, regex=True).sum(), FinancialServices2015["Region"].str.
  ↪contains("West", na=False, regex=True).sum()])

BankingNumberSouth = np.array([Banking1986["Region"].str.contains("South",␣
  ↪na=False, regex=True).sum(), Banking1996["Region"].str.contains("South",␣
  ↪na=False, regex=True).sum(), Banking2004["Region"].str.contains("South",␣
  ↪na=False, regex=True).sum(), Banking2010["Region"].str.contains("South",␣
  ↪na=False, regex=True).sum(), Banking2015["Region"].str.contains("South",␣
  ↪na=False, regex=True).sum()])
FinanceProductsNumberSouth = np.array([FinanceProducts1986["Region"].str.
  ↪contains("South", na=False, regex=True).sum(), FinanceProducts1996["Region"].
  ↪str.contains("South", na=False, regex=True).sum(),␣
  ↪FinanceProducts2004["Region"].str.contains("South", na=False, regex=True).
  ↪sum(), FinanceProducts2010["Region"].str.contains("South", na=False,␣
  ↪regex=True).sum(), FinanceProducts2015["Region"].str.contains("South",␣
  ↪na=False, regex=True).sum()])
FinancialServicesNumberSouth = np.array([FinancialServices1986["Region"].str.
  ↪contains("South", na=False, regex=True).sum(),␣
  ↪FinancialServices1996["Region"].str.contains("South", na=False, regex=True).
  ↪sum(), FinancialServices2004["Region"].str.contains("South", na=False,␣
  ↪regex=True).sum(), FinancialServices2010["Region"].str.contains("South",␣
  ↪na=False, regex=True).sum(), FinancialServices2015["Region"].str.
  ↪contains("South", na=False, regex=True).sum()])

BankingNumberEast = np.array([Banking1986["Region"].str.contains("East",␣
  ↪na=False, regex=True).sum(), Banking1996["Region"].str.contains("East",␣
  ↪na=False, regex=True).sum(), Banking2004["Region"].str.contains("East",␣
  ↪na=False, regex=True).sum(), Banking2010["Region"].str.contains("East",␣
  ↪na=False, regex=True).sum(), Banking2015["Region"].str.contains("East",␣
  ↪na=False, regex=True).sum()])
FinanceProductsNumberEast = np.array([FinanceProducts1986["Region"].str.
  ↪contains("East", na=False, regex=True).sum(), FinanceProducts1996["Region"].
  ↪str.contains("East", na=False, regex=True).sum(),␣
  ↪FinanceProducts2004["Region"].str.contains("East", na=False, regex=True).
  ↪sum(), FinanceProducts2010["Region"].str.contains("East", na=False,␣
  ↪regex=True).sum(), FinanceProducts2015["Region"].str.contains("East",␣
  ↪na=False, regex=True).sum()])
```

```
FinancialServicesNumberEast = np.array([FinancialServices1986["Region"].str.
 ↪contains("East", na=False, regex=True).sum(),␣
 ↪FinancialServices1996["Region"].str.contains("East", na=False, regex=True).
 ↪sum(), FinancialServices2004["Region"].str.contains("East", na=False,␣
 ↪regex=True).sum(), FinancialServices2010["Region"].str.contains("East",␣
 ↪na=False, regex=True).sum(), FinancialServices2015["Region"].str.
 ↪contains("East", na=False, regex=True).sum()])
```

[36]:
```
#This dataframe holds the business counts for food sector and the finance␣
 ↪sector, then works
#out the normalised difference from each of their respective 1986 business␣
 ↪count values

ComparativeDF = pd.DataFrame()
ComparativeDF["Year"] = [1986, 1996, 2004, 2010, 2015]
ComparativeDF["Food Sector Business Count"] = [len(Food1986), len(Food1996),␣
 ↪len(Food2004), len(Food2010), len(Food2015)]
ComparativeDF["Financial Sector Business Count"] = [len(Financial1986),␣
 ↪len(Financial1996), len(Financial2004), len(Financial2010),␣
 ↪len(Financial2015)]
ComparativeDF["Banking Sector Business Count"] = [len(Banking1986),␣
 ↪len(Banking1996), len(Banking2004), len(Banking2010), len(Banking2015)]
ComparativeDF["Difference (Food)"] = [0, (len(Food1996) / len(Food1986)) - 1,␣
 ↪(len(Food2004) / len(Food1986)) - 1, (len(Food2010) / len(Food1986)) -1,␣
 ↪(len(Food2015) / len(Food1986)) - 1]
ComparativeDF["Difference (Financial)"] = [0, (len(Financial1996) /␣
 ↪len(Financial1986)) - 1, (len(Financial2004) / len(Financial1986)) - 1,␣
 ↪(len(Financial2010) / len(Financial1986)) - 1, (len(Financial2015) /␣
 ↪len(Financial1986)) - 1]
ComparativeDF = ComparativeDF.set_index("Year")
```

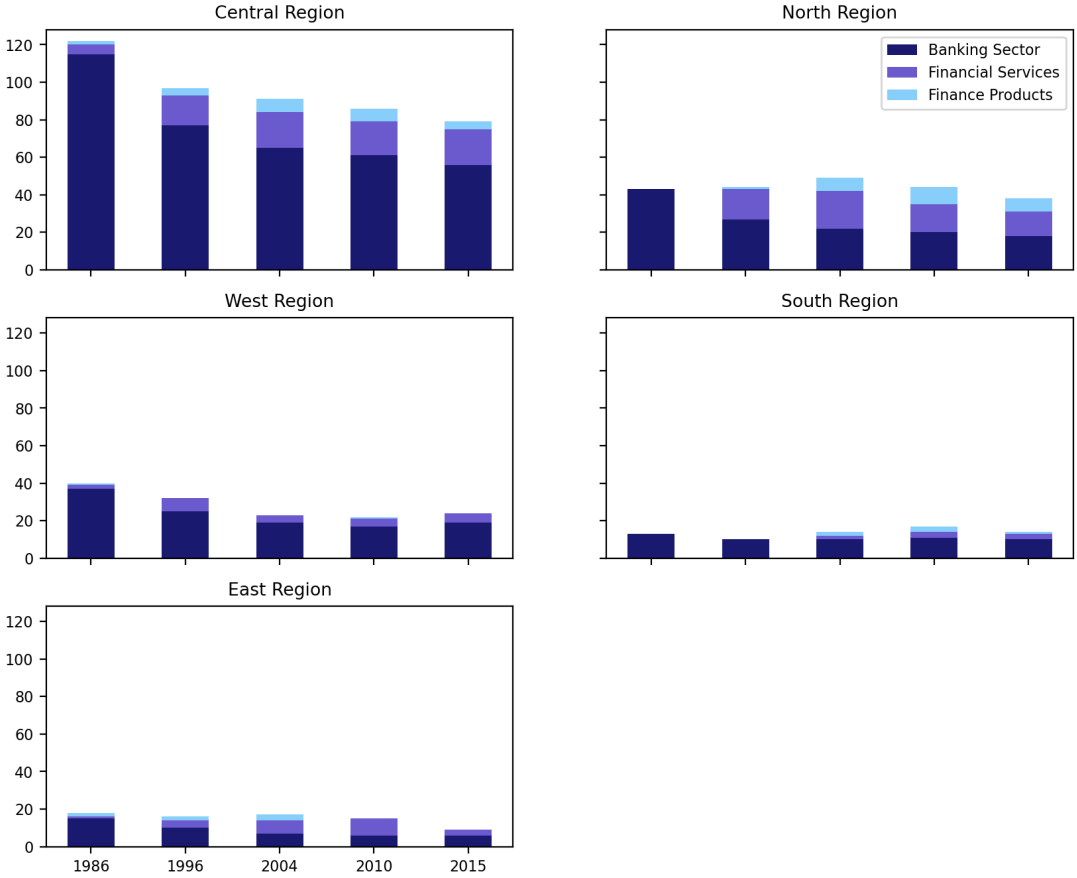
[37]:
```
years = [1986, 1996, 2004, 2010, 2015]
FinancialNumber = ComparativeDF["Financial Sector Business Count"]
FoodNumber = ComparativeDF["Food Sector Business Count"]

FinancialDifference = ComparativeDF["Difference (Financial)"]
FoodDifference = ComparativeDF["Difference (Food)"]
```

4 Data Communication and analysis

[38]:
```
#defines bar width and x axis
barWidth = 0.5
indices = np.arange(len(years))

fig, ax = plt.subplots(3, 2, sharex=True, sharey=True, figsize=(10, 8))
```

```
#enables sharing of x axis
plt.setp(ax, xticks=indices, xticklabels=years)
plt.delaxes(ax[2, 1])

#aggregate bar chart for the central region, created by adding each sub sector␣
 ↪onto the top of the other.
ax[0, 0].bar(height=BankingNumberCentral, x=indices, width=barWidth,␣
 ↪color="midnightblue")
ax[0, 0].bar(height=FinancialServicesNumberCentral, x=indices, width=barWidth,␣
 ↪bottom=BankingNumberCentral, color="slateblue")
ax[0, 0].bar(height=FinanceProductsNumberCentral, x=indices, width=barWidth,␣
 ↪bottom=BankingNumberCentral+FinancialServicesNumberCentral,␣
 ↪color="lightskyblue")
ax[0, 0].set_title("Central Region")

ax[0, 1].bar(height=BankingNumberNorth, x=indices, width=barWidth,␣
 ↪color="midnightblue", label="Banking Sector")
ax[0, 1].bar(height=FinancialServicesNumberNorth, x=indices, width=barWidth,␣
 ↪bottom=BankingNumberNorth, color="slateblue", label="Financial Services")
ax[0, 1].bar(height=FinanceProductsNumberNorth, x=indices, width=barWidth,␣
 ↪bottom=BankingNumberNorth+FinancialServicesNumberNorth,␣
 ↪color="lightskyblue", label="Finance Products")
ax[0, 1].set_title("North Region")

ax[1, 0].bar(height=BankingNumberWest, x=indices, width=barWidth,␣
 ↪color="midnightblue")
ax[1, 0].bar(height=FinancialServicesNumberWest, x=indices, width=barWidth,␣
 ↪bottom=BankingNumberWest, color="slateblue")
ax[1, 0].bar(height=FinanceProductsNumberWest, x=indices, width=barWidth,␣
 ↪bottom=BankingNumberWest+FinancialServicesNumberWest, color="lightskyblue")
ax[1, 0].set_title("West Region")

ax[1, 1].bar(height=BankingNumberSouth, x=indices, width=barWidth,␣
 ↪color="midnightblue")
ax[1, 1].bar(height=FinancialServicesNumberSouth, x=indices, width=barWidth,␣
 ↪bottom=BankingNumberSouth, color="slateblue")
ax[1, 1].bar(height=FinanceProductsNumberSouth, x=indices, width=barWidth,␣
 ↪bottom=BankingNumberSouth+FinancialServicesNumberSouth, color="lightskyblue")
ax[1, 1].set_title("South Region")

ax[2, 0].bar(height=BankingNumberEast, x=indices, width=barWidth,␣
 ↪color="midnightblue")
ax[2, 0].bar(height=FinancialServicesNumberEast, x=indices, width=barWidth,␣
 ↪bottom=BankingNumberEast, color="slateblue")
ax[2, 0].bar(height=FinanceProductsNumberEast, x=indices, width=barWidth,␣
 ↪bottom=BankingNumberEast+FinancialServicesNumberEast, color="lightskyblue")
```

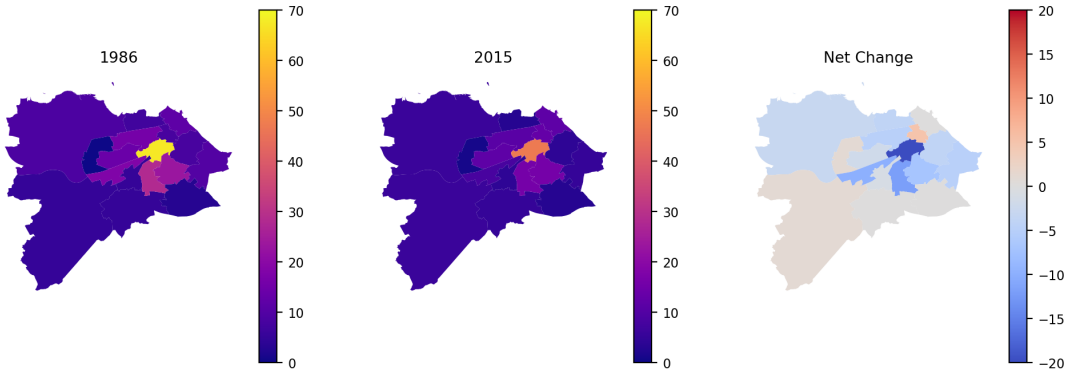
```
ax[2, 0].set_title("East Region")

ax[0, 1].legend()
plt.savefig("Finance_By_Region.png", dpi=200, bbox_inches='tight')
plt.show()
```



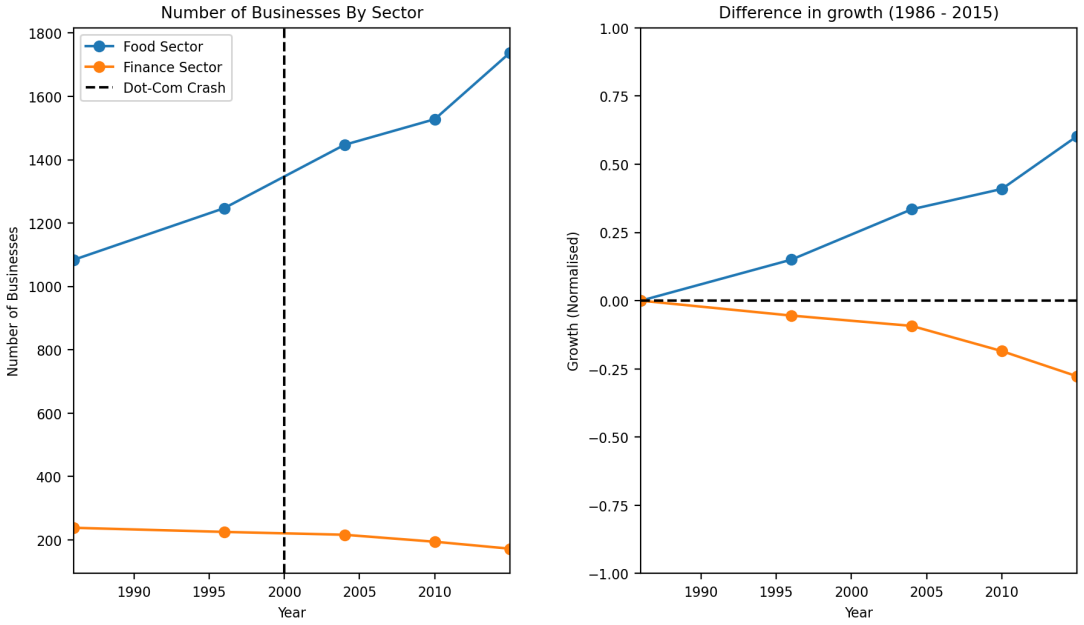
[39]:
```
ward_order = [
    "Almond",
    "Pentland Hills",
    "Drum Brae / Gyle",
    "Forth",
    "Inverleith",
    "Corstorphine / Murrayfield",
    "Sighthill / Gorgie",
    "Colinton / Fairmilehead",
    "Fountainbridge / Craiglockhart",
    "Morningside",
    "City Centre",
```

```
    "Leith Walk",
    "Leith",
    "Craigentinny / Duddingston",
    "Southside / Newington",
    "Liberton / Gilmerton",
    "Portobello / Craigmillar"
]
#counts the number of finance businesses in each ward for 1986 and 2015
counts86 = FinanceDF.loc[FinanceDF["Year"].isin([1986])]["Ward Name"].
 ↪value_counts()
counts15 = FinanceDF.loc[FinanceDF["Year"].isin([2015])]["Ward Name"].
 ↪value_counts()

#sets these to 0 for no value and the ward amount for each ward name in the list
Ward_Count86 = np.array([counts86.get(ward, 0) for ward in ward_order])
Ward_Count15 = np.array([counts15.get(ward, 0) for ward in ward_order])
```

[40]:
```
fig, ax = plt.subplots(1, 3, figsize=(12, 8))

#turns the x and y axis off
ax[0].set_axis_off()
ax[1].set_axis_off()
ax[2].set_axis_off()

#sets counts in each ward to the previously defined lists
wards["Count86"] = Ward_Count86
wards["Count15"] = Ward_Count15
#works out the difference between each of the 2
wards["Difference"] = np.subtract(Ward_Count15, Ward_Count86)

ax[0].set_title("1986")
ax[1].set_title("2015")
ax[2].set_title("Net Change")

wards.plot(ax=ax[0], column="Count86", legend=True, vmax=70, vmin=0,␣
 ↪cmap="plasma", legend_kwds={'shrink': 0.5})
wards.plot(ax=ax[1], column="Count15", legend=True, vmax=70, vmin=0,␣
 ↪cmap="plasma", legend_kwds={'shrink': 0.5})
wards.plot(ax=ax[2], column="Difference", legend=True,␣
 ↪vmin=min(wards["Difference"]), vmax=-(min(wards["Difference"])),␣
 ↪cmap="coolwarm", legend_kwds={'shrink': 0.5})
plt.savefig("Choropleth_Map.png", dpi=200, bbox_inches='tight')
```



[41]:
```
fig, ax = plt.subplots(1, 2, sharex=True, figsize=(10, 5))

#spaces out each of the subplots for readability
fig.tight_layout()
fig.subplots_adjust(left=0.15, wspace=0.3)

#defines axis for each of the plotd
ax[0].set_xticks(range(1985, 2015, 5))
ax[0].set_xlim(1986, 2015)
ax[0].set_title("Number of Businesses By Sector")
ax[0].set_xlabel("Year")
ax[0].set_ylabel("Number of Businesses")


ax[1].set_ylim(-1, 1)
ax[1].set_ylabel("Growth (Normalised)")
ax[1].set_title("Difference in growth (1986 - 2015)")
ax[1].set_xlabel("Year")

ax[0].plot(FoodNumber, marker="o", label="Food Sector")
ax[0].plot(FinancialNumber, marker="o", label="Finance Sector")

ax[1].plot(FoodDifference, marker="o")
ax[1].plot(FinancialDifference, marker="o")

ax[0].axvline(2000, linestyle="--", color="black", label="Dot-Com Crash")
ax[1].axhline(0, linestyle="--", color="black")

ax[0].legend()
plt.savefig("Comparative_Graph.png" ,dpi=200, bbox_inches='tight')
plt.show()
```



[42]:
```
yearsbefore = np.array([1986, 1996])
yearsafter = np.array([2004, 2010, 2015])
line_x = np.array([1985, 2015])
line_after_x = np.array([2000, 2015])

fig, ax = plt.subplots(figsize=(4, 4))

plt.ylim(-1, 1)
plt.xlim(1985, 2015)
plt.ylabel("Growth (Normalised)")
plt.title("Difference in growth (1986 - 2015)")
plt.xlabel("Year")

#createss a linear regression between the years and the financial difference␣
 ↪array in the comparativeDF
b, a = np.polyfit(x=yearsbefore, y=FinancialDifference[0:2], deg=1)
d, c = np.polyfit(x=yearsafter, y=FinancialDifference[2:], deg=1)

#plots initial regression line across entire graph, useful to work out where␣
 ↪the data "would" be if not for the crash

plt.plot(yearsbefore, FinancialDifference[0:2], "o")
plt.plot(yearsafter, FinancialDifference[2:], "o", color="tab:blue")
plt.axvline(2000, color="black", linestyle="--", label="Dot-Com Crash")
```

```
plt.plot(line_x, a + b * line_x, color="red", linestyle="--", label="Pre␣
 ↪Dot-Com Trend")
plt.plot(line_after_x, c + d * line_after_x, color="blue", linestyle="--",␣
 ↪label="Post Dot-Com Trend")

plt.legend()
plt.savefig("Regression.png" ,dpi=200, bbox_inches='tight')
plt.show()
```